

MapReduce + YARN Module

PART 1

Understanding MapReduce

Imagine we have a **huge log file**.

Example:

5 TB web logs

We want to calculate:

How many times each URL was visited

Example log:

```
/home  
/home  
/products  
/cart  
/products
```

Goal output:

```
/home 2  
/products 2  
/cart 1
```

Processing this on **one machine** would take hours.

Instead Hadoop processes the data **across many machines in parallel**.

This processing model is called **MapReduce**.

MapReduce is a **distributed programming model for large data processing**.

MapReduce

MapReduce Has Two Main Phases

Phase	Role
Map	transforms raw data into key-value pairs
Reduce	aggregates values by key

Simple Example

Input text

Hadoop is fast
Hadoop is scalable

Map Phase

Each word becomes a key-value pair.

(Hadoop,1)
(is,1)
(fast,1)
(Hadoop,1)
(is,1)
(scalable,1)

Shuffle Phase

Hadoop groups values by key.

(Hadoop,[1,1])
(is,[1,1])
(fast,[1])
(scalable,[1])

Reduce Phase

Reducer aggregates values.

(Hadoop,2)

(is,2)

(fast,1)

(scalable,1)

Explain clearly:

Map → Shuffle → Reduce

PART 2

How MapReduce Works Internally

Large input file:

1 GB file

HDFS block size:

128 MB

Blocks:

B1

B2

B3

B4

B5

B6

B7

B8

Each block becomes **one Map task**.

1 GB → 8 mappers

Each mapper processes one block independently.

This is why Hadoop scales very well.

Combiner Concept

Combiner performs **local aggregation** before sending data to reducers.

Example mapper output:

(Hadoop, 1)

(Hadoop, 1)

(Hadoop, 1)

Combiner converts it to:

(Hadoop, 3)

This reduces network traffic.

Combiner runs **on the same node as the mapper**.

MapReduce

Partitioner Concept

Partitioner decides:

which reducer receives which key

Default logic:

$\text{hash}(\text{key}) \% \text{number_of_reducers}$

Important rule:

Same keys must go to the same reducer

Otherwise results will be incorrect.

PART 3

Running a MapReduce Job (Lab)

We will run the **built-in Hadoop WordCount program**.

Step 1

Enter Hadoop container

```
docker exec -it namenode bash
```

Step 2

Create input file

```
echo "hadoop spark hadoop" > input.txt  
echo "spark flink hadoop" >> input.txt
```

View file:

```
cat input.txt
```

Output:

```
hadoop spark hadoop  
spark flink hadoop
```

Step 3

Upload file to HDFS

```
hdfs dfs -mkdir /mapreduce  
hdfs dfs -put input.txt /mapreduce
```

Verify:

```
hdfs dfs -ls /mapreduce
```

Step 4

Run WordCount MapReduce Job

Hadoop includes example programs.

Run:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar wordcount /mapreduce/input.txt /mapreduce/output
```

Hadoop will now start a MapReduce job.

```
map 0% reduce 0%  
map 100% reduce 100%
```

Step 5

Check Output

List output folder.

```
hdfs dfs -ls /mapreduce/output
```

View result.

```
hdfs dfs -cat /mapreduce/output/part-r-00000
```

Example output:

```
flink 1
hadoop 3
spark 2
```

Explain:

Reducers generated the final result

PART 4

Understanding YARN

Explain:

HDFS stores data

MapReduce processes data

But we also need **a system to manage resources**.

That system is **YARN**.

YARN manages cluster CPU and memory for applications.

Yarn

Think of YARN as:

Operating system of Hadoop cluster

YARN Architecture

Four main components:

Component

Role

ResourceManager	cluster resource manager
NodeManager	runs on each worker node
ApplicationMaster	manages a single job
Container	execution environment

These components coordinate job execution.

Yarn

Job Execution Flow

- 1 Client submits job
- 2 ResourceManager accepts job
- 3 ApplicationMaster starts
- 4 ApplicationMaster requests containers
- 5 NodeManagers launch tasks
- 6 Job finishes and resources released

YARN Job Execution Flow

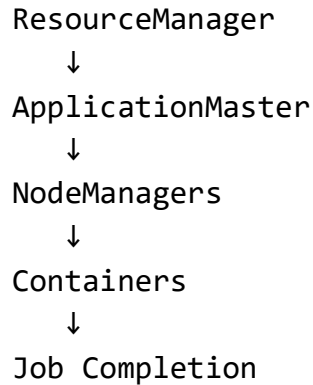
When a user runs a Hadoop job (for example a MapReduce job), the job does **not run directly on the cluster**.

Instead it is managed by **YARN**, which acts like the **cluster operating system**.

Full flow:

Client





We will go through **each step in detail**.

Step 1 — Client Submits the Job

The process starts when the user submits a job from the command line.

Example command:

```
hadoop jar hadoop-mapreduce-examples.jar wordcount input output
```

What happens internally:

1. Hadoop packages the job.
2. It sends job metadata to YARN.
3. Job files are uploaded to HDFS.

Files stored include:

```
job configuration
jar file
input location
output location
```

The client then sends a **job submission request** to the **ResourceManager**.

The client does not run the job itself.

It only submits the request to YARN.

Step 2 — ResourceManager Accepts the Job

The **ResourceManager (RM)** is the master component of YARN.

Its responsibilities:

job scheduling
resource allocation
cluster management

When the job arrives:

1. ResourceManager verifies the request.
2. It checks available resources in the cluster.
3. It decides where to launch the job coordinator.

This coordinator is called the **ApplicationMaster**.

ResourceManager **does NOT** run the job tasks directly.

Instead it only manages **resources and scheduling**.

Step 3 — Container Allocated for ApplicationMaster

Before the job can run, YARN must start an **ApplicationMaster (AM)**.

The ResourceManager allocates a **container** for this AM.

Container = execution unit.

A container includes:

CPU
Memory

Disk

Network

Think of a container like:

a reserved slot on a machine

Example:

Container 1

CPU: 2 cores

Memory: 4GB

The ResourceManager chooses a machine and asks its **NodeManager** to start the container.

Step 4 — ApplicationMaster Starts

Once the container is created, the **ApplicationMaster starts running**.

Each application/job has **its own ApplicationMaster**.

Example:

Job	ApplicationMaster
MapReduce job	AM1
Spark job	AM2
Hive query	AM3

Responsibilities of ApplicationMaster:

request resources

schedule tasks

monitor job execution

handle failures

So ApplicationMaster becomes the **manager of that specific job**.

Step 5 — ApplicationMaster Requests Containers for Tasks

Now the ApplicationMaster analyzes the job.

Example MapReduce job:

Input file = 1GB

Block size = 128MB

Number of blocks:

8 blocks

So the job requires:

8 mapper containers

ApplicationMaster sends a request to ResourceManager:

Request 8 containers for map tasks

Request 2 containers for reduce tasks

ResourceManager checks cluster resources and allocates containers.

Step 6 — NodeManagers Launch Containers

Each machine in the cluster runs a **NodeManager (NM)**.

NodeManager responsibilities:

start containers
monitor resource usage
send heartbeats to ResourceManager

When containers are assigned:

1. ResourceManager sends instructions to NodeManagers.
2. NodeManagers launch containers.
3. Containers execute the tasks.

Example execution:

Node1 → Mapper1

Node2 → Mapper2

Node3 → Mapper3

Tasks run **in parallel across machines**.

This is how Hadoop processes huge datasets quickly.

Step 7 — Map Tasks Execute

Each mapper reads **one block of input data**.

Example:

Input Block 1 → Mapper 1

Input Block 2 → Mapper 2

Each mapper generates **key-value pairs**.

Example mapper output:

(hadoop,1)

(spark,1)

(hadoop,1)

Mapper outputs are written to **local disk on the node**.

Step 8 — Shuffle and Sort Phase

After map phase finishes, Hadoop performs **shuffle and sort**.

This step:

groups values by key

Example:

Mapper outputs:

```
(hadoop,1)
(hadoop,1)
(spark,1)
```

Shuffle result:

```
(hadoop,[1,1])
(spark,[1])
```

Now data is ready for reducers.

Step 9 — Reduce Tasks Execute

Reducers perform aggregation.

Example reducer input:

```
(hadoop,[1,1])
```

Reducer output:

```
(hadoop,2)
```

Reducers write results back to **HDFS**.

Output example:

part-r-00000

Step 10 — Job Completion

When all tasks complete:

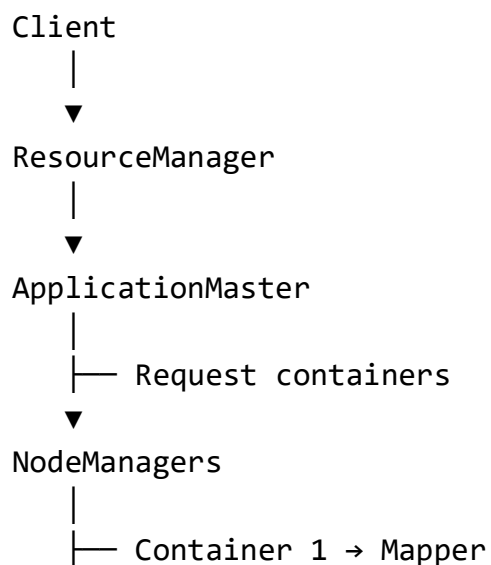
1. ApplicationMaster reports success to ResourceManager.
2. Containers are released.
3. Resources become available for new jobs.

Job status is updated.

Example status:

SUCCEEDED

Visual Diagram



└ Container 2 → Mapper
└ Container 3 → Reducer



Results written to HDFS

PART 5

YARN Lab

Open ResourceManager UI.

<http://localhost:8088>

Section	Meaning
Applications	running jobs
Nodes	cluster nodes
Containers	running tasks

View Running Applications

Run:

```
yarn application -list
```

Example output:

```
application_123456789
```

Kill Application

Run:

```
yarn application -kill <AppID>
```

Explain:

This stops a running job.

Exercise 1

- 1 create file
- 2 upload to HDFS
- 3 run wordcount job
- 4 check output

Exercise 2

Modify dataset:

```
hadoop hive spark spark  
spark flink flink
```

Predict output before running job.

Then run MapReduce.

Exercise 3

Run two MapReduce jobs simultaneously.

Observe YARN UI.

Explain how resources are shared.

Exercise 1

Predict number of mappers.

Dataset:

512MB file

block size = 128MB

Answer:

4 mappers

Exercise 2

If reducers = 2

Where will keys go?

partitioning.

Exercise 3

Submit two jobs simultaneously.

Observe YARN scheduler behavior.